Spring Boot Interview Prep

A study guide organized by topic, mapped to the code in this project. Each section lists **key concepts**, **common interview questions**, and **where to find the example in this codebase**.

Table of Contents

---

1. Spring Core (IoC, DI, Beans)

Key Concepts

- **Inversion of Control (IoC):** The framework controls object creation and lifecycle; you don't `new` your dependencies.
- **Dependency Injection (DI):** Objects receive their dependencies rather than initializing them (`new`). Spring injects via constructor, setter, or field.
- **IoC Container:** The `ApplicationContext` that manages beans (objects) — creates, wires, and destroys them.
- **Bean:** Any object managed by the Spring IoC container.
- **Bean Scopes:** singleton (default), prototype, request, session, application.
- **Stereotype Annotations:** `@Component` (generic), `@Service` (service layer), `@Repository` (DAO layer, adds exception translation), `@Controller`/`@RestController` (web layer).

Constructor Injection vs Field Injection

```java
// RECOMMENDED — constructor injection (this project uses this)
@RequiredArgsConstructor  // Lombok generates the constructor
public class UserService {
    private final UserRepository userRepository;  // final = immutable after injection
}

// DISCOURAGED — field injection
@Autowired
private UserRepository userRepository;
```

Why constructor injection is better: - Dependencies are visible in the constructor signature (clear API) - Fields can be `final` (immutable, thread-safe) - Easier to test without Spring (just pass mocks to the constructor) - Fails fast at startup if a dependency is missing

Common Questions

- **What is the difference between `@Component`, `@Service`, `@Repository`, `@Controller`?** Functionally identical (all are `@Component`). They mark the role/layer and `@Repository` adds automatic exception translation (checked SQL exceptions → unchecked `DataAccessException`).
- **Singleton vs prototype scope?** Singleton creates one instance per container; prototype creates a new instance every time it's requested.
- **`@Autowired` on constructor vs field?** Constructor injection is recommended. `@Autowired` is optional when there's only one constructor (Spring 4.3+).
- **What is the IoC container?** The `ApplicationContext` — it instantiates, configures, and assembles beans by reading configuration metadata (annotations/XML).
- **Can you have circular dependencies?** Yes, but it's discouraged. Spring resolves them via setter injection (constructor injection throws `BeanCurrentlyInCreationException`). Spring Boot 2.6+ disables circular references by default.
- **`@ComponentScan`:** How does Spring find your beans? The `@SpringBootApplication` annotation includes `@ComponentScan`, which scans the package and sub-packages of the main class.

Where in this project

- `UserService.java` — `@Service` + `@RequiredArgsConstructor` (constructor injection)
- `UserController.java` — `@RestController` + `@RequiredArgsConstructor`
- `UserRepository.java` — `@Repository`

---

2. Spring Boot Auto-Configuration

Key Concepts

- **Auto-configuration:** Spring Boot automatically configures beans based on what's on the classpath.
  - See H2 on classpath → configure in-memory H2 datasource.
  - See `spring-boot-starter-web` → start embedded Tomcat, configure `DispatcherServlet`.
  - See JPA on classpath → configure `EntityManagerFactory`, `DataSource`.
- **`@SpringBootApplication`:** Combines three annotations:

  - `@SpringBootConfiguration` — marks this as a configuration class (like `@Configuration`)
  - `@EnableAutoConfiguration` — triggers auto-configuration
  - `@ComponentScan` — scans for components in this package and below
- **`@EnableAutoConfiguration`:** Uses `META-INF/spring/org.springframework.boot.autoconfigure.AutoConfiguratio` file (Spring Boot 3.x) to list auto-configuration classes.
- **Conditional annotations:**
  - `@ConditionalOnClass` — only configures if a class is on the classpath
  - `@ConditionalOnMissingBean` — only configures if you haven't already defined this bean
  - `@ConditionalOnProperty` — only configures if a property is set
  - `@ConditionalOnBean` / `@ConditionalOnWebApplication`

Common Questions

- **How does auto-configuration work?** Spring Boot scans the classpath and conditional annotations to decide which beans to create. The `spring-boot-autoconfigure` JAR contains hundreds of auto-configuration classes, most guarded by `@ConditionalOnClass`.
- **How to disable a specific auto-configuration?**

```
@SpringBootApplication(exclude = {DataSourceAutoConfiguration.class})
```

- **What is the difference between `@Configuration` and `@Component`?** `@Configuration` is a specialized `@Component` where `@Bean` methods produce CGLIB-proxied beans (singleton by default). `@Component` with `@Bean` methods calls them directly (no proxy, so not singleton).
- **What is `spring-boot-starter-parent`?** A POM that provides default dependency versions, plugin configurations, and Java version. It's the "BOM" (Bill of Materials) for Spring Boot projects.

Where in this project

- `DemoApplication.java` — `@SpringBootApplication`
- `pom.xml` — `spring-boot-starter-parent` and starters
- The app auto-configured Tomcat (embedded server), H2 (datasource), JPA (Hibernate) — all visible in the startup logs

---

3. Spring MVC & REST Controllers

Key Concepts

- **`@RestController`:** Combines `@Controller` + `@ResponseBody`. Returns data (JSON) directly, not a view name.
- **`@ResponseBody`:** Tells Spring to serialize the return value into the HTTP response body (using Jackson by default).
- **`@RequestMapping` / HTTP method shortcuts:**
  - `@GetMapping`, `@PostMapping`, `@PutMapping`, `@DeleteMapping`, `@PatchMapping`
- **`@PathVariable`:** Extracts values from the URI (`/users/{id}` → `id`)
- **`@RequestParam`:** Extracts query parameters (`/users?email=x` → `email`)
- **`@RequestBody`:** Deserializes the HTTP request body into a Java object (JSON → POJO via Jackson)
- **`@ResponseStatus`:** Sets the HTTP status code for a response
- **`ResponseEntity<T>`:** Full control over HTTP response — status, headers, and body

Request Flow (DispatcherServlet)

```
HTTP Request
    → DispatcherServlet (front controller)
        → HandlerMapping (finds the right controller method)
        → HandlerAdapter (invokes the method)
        → Controller returns data
        → HttpMessageConverter (Jackson serializes to JSON)
    → HTTP Response
```

Common Questions

- **`@Controller` vs `@RestController`?** `@RestController` = `@Controller` + `@ResponseBody`. `@Controller` returns view names (for SSR templates like Thymeleaf); `@RestController` returns data directly.
- **`@PathVariable` vs `@RequestParam`?** `@PathVariable` extracts from the URL path (`/users/123`); `@RequestParam` extracts from query string (`/users?id=123`).
- **How does JSON serialization work?** Jackson's `HttpMessageConverter` automatically converts Java objects to JSON and vice versa. `@ResponseBody` triggers serialization; `@RequestBody` triggers deserialization.
- **What is `ResponseEntity` and when to use it?** When you need to set custom HTTP status codes or headers. For simple cases, `@ResponseStatus` or returning the object directly is fine.
- **Content negotiation?** Spring can serve different formats (JSON, XML) based on the `Accept` header. Configure via `ContentNegotiationManager`.

Where in this project

- `UserController.java` — full CRUD REST API with `@GetMapping`, `@PostMapping`, `@PutMapping`, `@DeleteMapping`
- `HelloController.java` — simple `@GetMapping` with `@RequestParam`
- `UserController.createUser()` — uses `ResponseEntity.created(location)` to return HTTP 201 with `Location` header

---

4. Validation (Bean Validation)

Key Concepts

- **Jakarta Bean Validation (formerly javax.validation):** A spec for declarative validation using annotations.
- **Hibernate Validator:** The reference implementation (bundled with `spring-boot-starter-validation`).
- **Common constraints:**
  - `@NotBlank` — not null and not empty (strings)
  - `@NotNull` — not null (any type)
  - `@NotEmpty` — not null and not empty (collections, strings, maps)
  - `@Email` — valid email format
  - `@Size(min, max)` — length constraint
  - `@Min` / `@Max` — numeric range
  - `@Pattern(regexp)` — regex match

  - `@Positive` / `@Negative` — numeric sign

- **`@Valid`:** Triggers validation on a `@RequestBody` parameter. Spring validates before the method executes.
- **`@Validated`:** Class-level annotation for method-level validation (e.g., validating `@RequestParam` values).
- **`MethodArgumentNotValidException`:** Thrown when `@Valid` fails on a `@RequestBody`. Caught in the global exception handler.

Common Questions

- **`@Valid` vs `@Validated`?** `@Valid` is from Jakarta Validation, used on parameters/fields for cascading validation. `@Validated` is Spring's extension, supports validation groups and method-level validation.
- **`@NotBlank` vs `@NotNull` vs `@NotEmpty`?**
  - `@NotNull`: not null (can be empty string)
  - `@NotEmpty`: not null and size > 0
  - `@NotBlank`: not null and contains at least one non-whitespace character
- **How does validation integrate with Spring MVC?** When `@Valid` is on a `@RequestBody` parameter, the `RequestResponseBodyMethodProcessor` validates the object before the controller method runs. If validation fails, it throws `MethodArgumentNotValidException` before your code runs.
- **Validation groups?** You can group constraints (e.g., `OnCreate`, `OnUpdate`) and validate only specific groups using `@Validated(OnCreate.class)`.

Where in this project

- `UserRequest.java` — `@NotBlank`, `@Email`, `@Size`, `@Min` constraints
- `User.java` — entity-level validation constraints
- `UserController.java` — `@Valid` on `@RequestBody` parameters
- `GlobalExceptionHandler.java` — `handleValidation()` catches `MethodArgumentNotValidException` and returns field-specific error details

---

5. Exception Handling

Key Concepts

- **`@ControllerAdvice` / `@RestControllerAdvice`:** Global exception handling across all controllers. `@RestControllerAdvice` = `@ControllerAdvice` + `@ResponseBody`.
- **`@ExceptionHandler`:** Maps an exception type to a handler method.
- **`ResponseEntityExceptionHandler`:** Base class with built-in handlers for standard Spring MVC exceptions.
- **Custom exceptions:** Extend `RuntimeException` for specific business error cases.
- **Error response format:** A structured JSON body (timestamp, status, error, message, path) — similar to Spring Boot's default error response but customizable.

Common Questions

- **How does `@ControllerAdvice` work?** It's a component that wraps all controllers. When any controller throws an exception, Spring's `ExceptionHandlerExceptionResolver` checks if a matching `@ExceptionHandler` exists in the `@ControllerAdvice` beans.

- **`@ControllerAdvice` vs `@RestControllerAdvice`?** `@RestControllerAdvice` adds `@ResponseBody` automatically so the handler return value is serialized to JSON. With `@ControllerAdvice` you'd need `@ResponseBody` on each method or return `ResponseEntity`.
- **What happens if no exception handler matches?** Spring Boot's `BasicErrorController` handles it → forwards to `/error` → renders the default error JSON (or the error page).
- **How to handle different exceptions differently?** Create multiple `@ExceptionHandler` methods, one per exception type. Spring picks the most specific match.
- **Should you catch `Exception` globally?** Yes, as a catch-all to prevent stack traces leaking to the client. Return a generic 500 response and log the details server-side.

Where in this project

- `GlobalExceptionHandler.java` — handles `ResourceNotFoundException` (404), `EmailAlreadyExistsException` (409), `MethodArgumentNotValidException` (400), and generic `Exception` (500)
- `ResourceNotFoundException.java`, `EmailAlreadyExistsException.java` — custom exceptions
- `ErrorResponse.java` — structured error response body with `@JsonInclude(NON_NULL)` to omit null fields

---

6. Spring Data JPA

Key Concepts

- **Spring Data JPA:** A Spring project that reduces JPA boilerplate. You write interfaces; Spring generates implementations at runtime.
- **`JpaRepository<T, ID>`:** The main repository interface. Provides CRUD, pagination, and sorting out of the box.
- **Repository hierarchy:** `Repository` → `CrudRepository` → `JpaRepository` (→ `PagingAndSortingRepository` in between)
- **Query methods:** Spring parses method names and generates SQL:
  - `findByEmail(String email)` → `SELECT ... WHERE email = ?`
  - `existsByEmail(String email)` → `SELECT COUNT(*) > 0 ... WHERE email = ?`
  - `findByEmailAndAge(String email, Integer age)` → `WHERE email = ? AND age = ?`
  - `findByOrderByAgeDesc()` → `ORDER BY age DESC`
- **`@Query`:** Custom JPQL or native SQL queries

```java
@Query("SELECT u FROM User u WHERE u.age > :age")
List<User> findOlderThan(@Param("age") Integer age);
```

- **`@Modifying`:** Marks a `@Query` as an UPDATE/DELETE (requires `@Transactional`)
- **Named queries, `@NamedQuery`, `@NamedNativeQuery`**
- **Pagination:** `Pageable` parameter → returns `Page<T>` with content + metadata (total pages, total elements)
- **Sorting:** `Sort` parameter or `Pageable` with sort
- **Projections:** Interface-based or DTO-based projections to fetch partial data

Common Questions

- **How does Spring Data JPA generate implementations?** At startup, Spring scans repository interfaces and creates a JDK dynamic proxy for each. The proxy delegates to `SimpleJpaRepository` (default implementation) which uses the `EntityManager`.
- **`findById` vs `getById`?** `findById` returns `Optional<T>` and executes the query immediately. `getById` returns a lazy proxy (no DB hit until a field is accessed) — useful for setting foreign-key references without loading the entity.
- **What is the difference between `save()` and `merge()`?** `save()` checks if the entity is new (no ID) → persist; if existing → merge. Spring Data's `save()` delegates to `persist()` for new entities and `merge()` for detached entities.
- **Derived query method naming conventions?** `find{SubjectBy}{Predicate}` — e.g., `findByLastNameAndAgeGreaterThanOrderByIdDesc(String lastName, Integer age)`.
- **`@Query` JPQL vs native SQL?** JPQL works with entity names and entity fields (portable). Native SQL uses actual table/column names (not portable, but can use DB-specific features).
- **N+1 query problem?** When fetching a list of entities with lazy relationships, each access to the relationship triggers a separate query. Solutions: `@EntityGraph`, `JOIN FETCH`, or `@BatchSize`.

Where in this project

- `UserRepository.java` — extends `JpaRepository<User, Long>`, uses derived query methods `findByEmail` and `existsByEmail`
- `UserService.java` — calls `findAll()`, `findById()`, `save()`, `deleteById()`, `existsById()`, `existsByEmail()`

---

7. Hibernate & ORM Concepts

Key Concepts

- **JPA (Jakarta Persistence API):** The specification. Hibernate is the implementation (default in Spring Boot).
- **`@Entity`:** Marks a class as a persistent entity (mapped to a DB table).
- **`@Table`:** Specifies the table name (defaults to class name).
- **`@Id` + `@GeneratedValue`:** Primary key + auto-generation strategy.
- **Generation strategies:**
  - `IDENTITY` — DB auto-increment column (most common for MySQL/PostgreSQL)
  - `SEQUENCE` — uses a DB sequence (default for Oracle)
  - `TABLE` — uses a separate table to simulate a sequence
  - `AUTO` — provider picks the strategy
- **`@Column`:** Column mapping with constraints (nullable, unique, length)
- **Entity lifecycle states:**
  - **New/Transient:** Not yet persisted, no ID
  - **Managed/Persistent:** Attached to persistence context, tracked for changes
  - **Detached:** Was managed, now disconnected (e.g., after transaction close)
  - **Removed:** Marked for deletion
- **Persistence context (first-level cache):** The set of managed entities in a session. Changes to managed entities are automatically flushed.
- **Dirty checking:** Hibernate compares the current state of managed entities with their original state. If different, it generates an UPDATE on flush.

- **`ddl-auto` values:** `none`, `validate`, `update`, `create`, `create-drop`
  - `update` — update schema if entity changes (dev only!)
  - `create` — drop and recreate on every start (wipes data)
  - `validate` — only check schema matches entities (prod)
  - `none` — do nothing (prod)
- **Lazy vs Eager loading:**
  - `@ManyToOne`, `@OneToOne` → EAGER by default
  - `@OneToMany`, `@ManyToMany` → LAZY by default
- **`@Transactional` and the persistence context:** Within a transaction, `save()` returns a managed entity. Changes to it are auto-flushed at commit.

Common Questions

- **What is the difference between JPA and Hibernate?** JPA is the spec (API); Hibernate is one implementation. Spring Data JPA is a Spring wrapper that reduces boilerplate.
- **What is the first-level cache?** The persistence context — a per-transaction map of managed entities. Prevents duplicate queries within the same transaction.
- **What is the second-level cache?** A cross-session cache (shared). Needs explicit configuration (e.g., Ehcache, Caffeine). Disabled by default.
- **Dirty checking?** Hibernate tracks changes to managed entities and generates UPDATE statements automatically — you don't need to call `save()`.
- **`persist` vs `merge`?** `persist` adds a new entity to the persistence context (must not have an ID). `merge` copies the state of a detached entity into a managed entity and returns the managed copy.
- **`@Version` and optimistic locking?** A `@Version` field (int/timestamp) is checked on update. If the version in the DB differs (someone else updated first), `OptimisticLockException` is thrown.
- **`@EntityGraph`?** Solves the N+1 problem by specifying which associations to fetch eagerly for a specific query.

Where in this project

- `User.java` — `@Entity`, `@Table(name = "users")`, `@Id`, `@GeneratedValue(strategy = IDENTITY)`, `@Column(nullable, unique)`
- `application.yml` — `spring.jpa.hibernate.ddl-auto: update`, `spring.jpa.show-sql: true`
- Startup logs show Hibernate creating the `users` table and the unique email constraint

---

8. Transaction Management

Key Concepts

- **`@Transactional`:** Declares a method/class should run in a transaction. Spring creates a proxy that begins a transaction before the method and commits (or rolls back) after.
- **Propagation types** (what happens if a transaction already exists):
  - `REQUIRED` (default) — join existing, or start new
  - `REQUIRES_NEW` — always start a new, suspend existing
  - `NESTED` — nested transaction (savepoint)
  - `SUPPORTS` — join if exists, otherwise run non-transactional
  - `NOT_SUPPORTED` — run non-transactional, suspend existing

  - `MANDATORY` — must be called within a transaction
  - `NEVER` — must not be called within a transaction
- **Isolation levels:**
  - `DEFAULT` — use DB default
  - `READ_UNCOMMITTED` — can read uncommitted data (dirty reads)
  - `READ_COMMITTED` — only committed data (no dirty reads)
  - `REPEATABLE_READ` — same query returns same results in same txn
  - `SERIALIZABLE` — full locking, no concurrent modifications
- **Rollback rules:**
  - By default, rolls back on unchecked exceptions (`RuntimeException`, `Error`)
  - Does NOT roll back on checked exceptions unless you specify `rollbackFor`
  - `noRollbackFor` — don't roll back for specific exceptions
- **`readOnly = true`:** Hint to the persistence provider that no writes will happen. Can enable optimizations.
- **Self-invocation problem:** Calling a `@Transactional` method from another method in the same class does NOT start a transaction (the proxy is bypassed).

Common Questions

- **How does `@Transactional` work under the hood?** Spring creates a CGLIB or JDK proxy around the bean. When you call the method, the proxy intercepts, begins a transaction via `PlatformTransactionManager`, invokes the method, and commits or rolls back.
- **Does `@Transactional` roll back on checked exceptions?** No, only on unchecked (`RuntimeException`) and `Error`. Use `@Transactional(rollbackFor = Exception.class)` to include checked exceptions.
- **What is the self-invocation problem?** If method A (not transactional) calls method B (`@Transactional`) in the same class, B runs without a transaction because the call bypasses the proxy (`this.b()` not `proxy.b()`).
- **`REQUIRED` vs `REQUIRES_NEW`?** `REQUIRED` joins the existing transaction. `REQUIRES_NEW` suspends the outer transaction and starts a completely independent one (useful for audit logging that should succeed even if the main txn rolls back).
- **Should `@Transactional` be on the service or controller layer?** Service layer. Controllers shouldn't manage transactions; services encapsulate business logic.
- **`readOnly = true` — when to use?** On read-only service methods. The DB can optimize, and Hibernate skips dirty checking. This project uses it on `getAllUsers()`, `getUserById()`, `getUserByEmail()`.

Where in this project

- `UserService.java` — `@Transactional` at class level (all methods transactional); `@Transactional(readOnly = true)` on read methods
- Write methods (`createUser`, `updateUser`, `deleteUser`) use the default (read-write) propagation

---

9. DTO Pattern & Layered Architecture

Key Concepts

- **Layered architecture:**
  - **Controller layer** — handles HTTP, delegates to service

  - **Service layer** — business logic, transaction boundaries
  - **Repository layer** — data access (Spring Data JPA interfaces)
  - **Model/Entity layer** — JPA entities (database representation)
- **DTO (Data Transfer Object):** Objects designed for the API layer, decoupled from the entity model.
  - **Request DTO:** What the client sends (e.g., `UserRequest`) — no `id`, has validation
  - **Response DTO:** What the client receives (e.g., `UserResponse`) — has `id`, may omit internal fields
- **Why DTOs instead of returning entities directly?**
  - Prevents over-exposure of sensitive or internal fields
  - Decouples API contract from DB schema (you can change one without the other)
  - Allows different validation rules for input vs the entity
  - Avoids lazy-loading serialization issues (Jackson may trigger N+1 queries when serializing entities)
- **Mapping:** Manual mapping (as in this project) or libraries like **MapStruct**, **ModelMapper**, **MapStruct**.

Common Questions

- **Why use DTOs?** Decouple API from DB schema, control what's exposed, avoid serialization issues, apply input-specific validation.
- **DTO vs Entity?** Entity maps to a DB table; DTO maps to an API request/response. They serve different purposes and should be separate.
- **How do you map entity to DTO?** Manual mapping (simple, explicit), MapStruct (compile-time generated mappers), ModelMapper (reflection-based), or Jackson views (`@JsonView`).
- **What is the Open-Closed Principle in layered architecture?** Each layer should be open for extension but closed for modification. The service layer should depend on abstractions, not concrete repositories.

Where in this project

- `UserRequest.java` — request DTO with validation constraints (no `id`)
- `UserResponse.java` — response DTO (includes `id`)
- `UserService.java` — `toResponse()` method maps `User` entity → `UserResponse` DTO
- `UserController.java` — accepts `UserRequest`, returns `UserResponse` (never exposes the entity directly)

---

10. Configuration & Externalized Properties

Key Concepts

- **Externalized configuration:** Spring Boot lets you put config outside the JAR — `application.properties/.yml`, command-line args, env vars, etc.
- **`@Value`:** Inject a single property value

```java
@Value("${server.port}")
private int port;
```

- **`@ConfigurationProperties`:** Bind a group of properties to a POJO (type-safe, validated)

```java
@ConfigurationProperties(prefix = "app")
@Component
public class AppProperties {
    private String name;
    private int maxRetries;
}
```

- **`@Configuration`:** Marks a class as a source of bean definitions (contains `@Bean` methods)
- **`@Bean`:** Declares a bean that Spring should manage (used in `@Configuration` classes)
- **Property sources (in order of precedence, highest first):**
  1. Command-line args
  2. System properties
  3. OS environment variables
  4. `application-{profile}.yml` (profile-specific)
  5. `application.yml` / `application.properties` (default)
- **`@PropertySource`:** Load a specific `.properties` file (does not work with YAML)

Common Questions

- **`@Value` vs `@ConfigurationProperties`?** `@Value` is for single values; `@ConfigurationProperties` binds a hierarchy to a POJO (type-safe, supports validation, IDE auto-completion).
- **YAML vs properties?** YAML supports hierarchical data (cleaner for nested config); `.properties` is flat key-value. Both work with Spring Boot.
- **How to override properties at runtime?** Command-line args (`--server.port=9090`), env vars (`SERVER_PORT=9090`), or a config server (Spring Cloud Config).
- **What is `@Configuration` and how is it different from `@Component`?** `@Configuration` is a `@Component` whose `@Bean` methods are CGLIB-proxied to ensure singleton semantics. `@Component` with `@Bean` methods would call the method directly, producing a new instance each time.
- **What is `@Bean`?** A method-level annotation that registers the return value as a bean in the Spring container. Used in `@Configuration` classes for third-party libraries you can't annotate.

Where in this project

- `application.yml` — `server.port`, `spring.datasource.*`, `spring.jpa.*`, `spring.h2.console.*`
- Could add `@ConfigurationProperties` classes for custom properties

---

11. Profiles & Environment

Key Concepts

- **Profiles:** A way to group configuration for different environments (dev, test, prod).
- **`@Profile("dev")`:** Bean or `@Configuration` is only active when the given profile is enabled.
- **`spring.profiles.active`:** Property to activate profiles.
- **Profile-specific files:** `application-dev.yml`, `application-prod.yml` — loaded when the profile is active.
- **`@ConfigurationProperties` with profiles:** Different config bindings per environment.
- **`Environment` abstraction:** `Environment` bean gives access to active profiles and properties.

Common Questions

- **How do you activate a profile?** `spring.profiles.active=dev` in `application.yml`, or `--spring.profiles.active=dev` on the command line, or `SPRING_PROFILES_ACTIVE=dev` env var.
- **Can you have multiple active profiles?** Yes: `spring.profiles.active=dev,debug`.
- **`@Profile` on a `@Bean` method?** Only creates the bean if the profile is active. Useful for environment-specific beans (e.g., a mock vs real payment client).
- **Default profile?** If no profile is active, Spring uses "default" profile. `application-default.yml` would be loaded.

---

12. Actuator & Monitoring

Key Concepts

- **Spring Boot Actuator:** Adds production-ready endpoints for monitoring and managing the app.
- **Add dependency:** `spring-boot-starter-actuator`
- **Key endpoints:**
  - `/actuator/health` — app health check (used by Kubernetes liveness/readiness probes)
  - `/actuator/info` — app info (custom properties)
  - `/actuator/metrics` — JVM, HTTP, and custom metrics
  - `/actuator/loggers` — view and change log levels at runtime
  - `/actuator/env` — environment properties
  - `/actuator/beans` — all registered beans
  - `/actuator/mappings` — all URL mappings
- **Exposure:** By default, only `health` and `info` are exposed over HTTP. Use `management.endpoints.web.exposure.incl` to expose all.
- **Security:** Secure actuator endpoints with Spring Security.
- **Integration with Micrometer/Prometheus/Grafana** for metrics dashboards.

Common Questions

- **What is Actuator?** A Spring Boot sub-project that adds operational endpoints (health, metrics, env, etc.) to your application.
- **How to secure actuator endpoints?** Spring Security + role-based access. Or restrict which endpoints are exposed via `management.endpoints.web.exposure`.
- **What is `management.endpoints.web.exposure.include`?** Controls which endpoints are accessible over HTTP. Default: `health,info`.
- **Custom health indicator?** Implement `HealthIndicator` and override `health()`.

---

13. Testing in Spring Boot

Key Concepts

- **Test slices (narrow context):**
  - `@WebMvcTest` — loads only the web layer (controllers, MockMvc). No service/repo/DB. Fast.
  - `@DataJpaTest` — loads only JPA layer (repositories, H2). No web. Fast.

  - `@JsonTest` — tests JSON serialization/deserialization.

- **Full context:**
  - `@SpringBootTest` — loads the entire application context. Slower but most realistic.
  - `@SpringBootTest(webEnvironment = RANDOM_PORT)` — starts a real server on a random port.

- **`MockMvc`:** Mock the servlet container for testing controllers without a real server.
- **`@MockBean`:** Replaces a bean with a Mockito mock in the Spring context.
- **`@TestConfiguration`:** Provides test-specific beans.
- **`@Transactional` in tests:** Rolls back at the end of each test (no DB pollution).
- **`@Sql`:** Load SQL scripts before a test.
- **Testcontainers:** Spin up real Docker containers (PostgreSQL, etc.) for integration tests.

Common Questions

- **`@SpringBootTest` vs `@WebMvcTest`?** `@SpringBootTest` loads everything (slow, integration). `@WebMvcTest` loads only the web layer with `MockMvc` (fast, focused).
- **How to test a controller?** `@WebMvcTest(YourController.class)` + `@MockBean` for the service + `MockMvc` to perform requests and assert responses.
- **How to test a repository?** `@DataJpaTest` — uses an embedded H2 by default, auto-configures `TestEntityManager`, rolls back after each test.
- **`@MockBean` vs `@Mock`?** `@MockBean` replaces a bean in the Spring context (Spring integration). `@Mock` is pure Mockito (no Spring context, used in unit tests).
- **What is `TestEntityManager`?** A JPA helper for tests provided by Spring Boot — simpler than `EntityManager` for test setup.

Where in this project

- `DemoApplicationTests.java` — `@SpringBootTest` context load test (smoke test)
- To add: `UserControllerTest` (`@WebMvcTest`), `UserServiceTest` (unit test with `@Mock`), `UserRepositoryTest` (`@DataJpaTest`)

---

14. Security (Spring Security)

Key Concepts

- **Spring Security:** Authentication (who are you?) + Authorization (what can you do?).
- **Filter chain:** Spring Security is implemented as a servlet filter chain (`DelegatingFilterProxy` → `FilterChainProxy`).
- **Authentication:**
  - Basic Auth, Form login, JWT, OAuth2
  - `AuthenticationManager`, `AuthenticationProvider`, `UserDetailsService`
- **Authorization:**
  - `@PreAuthorize("hasRole('ADMIN')")`, `@PostAuthorize`
  - `SecurityFilterChain` bean (Spring Security 6 / Spring Boot 3)
- **CSRF, CORS:** Security headers and cross-origin configuration.
- **Stateless (JWT) vs Stateful (session) auth.**

Common Questions

- **How does Spring Security work?** A filter chain intercepts every request. Each filter handles a concern (authentication, authorization, CSRF, etc.). The `SecurityFilterChain` bean configures which filters apply to which URLs.
- **What is a `SecurityFilterChain`?** Spring Security 6 replaced `WebSecurityConfigurerAdapter` with a `SecurityFilterChain` bean:

```
@Bean
public SecurityFilterChain filterChain(HttpSecurity http) throws Exception {
    http.authorizeHttpRequests(auth -> auth
            .requestMatchers("/api/public/**").permitAll()
            .anyRequest().authenticated())
        .httpBasic(Customizer.withDefaults());
    return http.build();
}
```

- **What is JWT?** JSON Web Token — a self-contained token (header.payload.signature). Stateless auth: the server doesn't store sessions; the client sends the token in the `Authorization: Bearer <token>` header.
- **`@PreAuthorize` vs `@Secured`?** `@PreAuthorize` supports SpEL expressions (richer: `hasRole('ADMIN') and #user.id == authentication.principal.id`). `@Secured` only supports role names.
- **How to secure a REST API?** JWT for stateless auth, HTTPS, CORS config, input validation, rate limiting, and role-based authorization.

---

15. Database & Connection Pooling

Key Concepts

- **Connection pool:** A pool of reusable DB connections. Avoids the cost of opening/closing a connection per request.
- **HikariCP:** The default connection pool in Spring Boot (fastest). Key settings:
  - `maximum-pool-size` (default: 10)
  - `minimum-idle`
  - `connection-timeout` (default: 30s)
  - `idle-timeout`
- **Flyway / Liquibase:** Database migration tools. Version-controlled schema changes.
  - `spring.flyway.enabled=true` + `db/migration/V1__init.sql`
- **`@EntityScan` / `@EnableJpaRepositories`:** Customize where Spring looks for entities/repositories.
- **Multiple datasources:** Configure with `@ConfigurationProperties` + `DataSourceBuilder`.

Common Questions

- **Why use a connection pool?** Opening a DB connection is expensive (TCP handshake, auth). A pool reuses connections, dramatically improving throughput.
- **What is HikariCP?** The default connection pool in Spring Boot. Known for performance and reliability.

- **Flyway vs Liquibase?** Flyway uses SQL scripts (simpler). Liquibase uses XML/YAML/JSON changelogs (DB-agnostic, more features).
- **How to handle multiple datasources?** Define multiple `DataSource` beans with `@ConfigurationProperties`, and `@EnableJpaRepositories` per datasource with `entityManagerFactoryRef` and `transactionManagerRef`.

Where in this project

- `application.yml` — H2 in-memory datasource config. HikariCP is auto-configured (visible in startup logs: `HikariPool-1 - Starting...`)
- H2 console at `/h2-console` for inspecting the in-memory DB

---

16. Caching

Key Concepts

- **`@Cacheable`:** Checks cache before method execution; if found, returns cached value without executing the method.
- **`@CachePut`:** Always executes the method and puts the result in the cache (for updates).
- **`@CacheEvict`:** Removes entries from the cache (for deletes).
- **`@EnableCaching`:** Enables annotation-based caching on a `@Configuration` class.
- **Cache providers:** Caffeine (in-memory, recommended), Ehcache, Redis (distributed), Hazelcast.
- **Cache names and keys:** `@Cacheable(value = "users", key = "#id")`

Common Questions

- **`@Cacheable` vs `@CachePut`?** `@Cacheable` skips the method if cached. `@CachePut` always executes and updates the cache.
- **What is a cache hit/miss?** Hit: found in cache, method skipped. Miss: not in cache, method executes, result cached.
- **Local vs distributed cache?** Local (Caffeine, Ehcache) — fast, per-instance, not shared. Distributed (Redis, Hazelcast) — shared across instances, slower but consistent.
- **Cache invalidation strategies?** TTL (time-to-live), explicit eviction (`@CacheEvict`), write-through, write-behind.

---

17. Async & Scheduling

Key Concepts

- **`@EnableAsync`:** Enables async method execution.
- **`@Async`:** Method runs in a separate thread (returns `CompletableFuture` or `void`).
- **`@EnableScheduling`:** Enables scheduled tasks.
- **`@Scheduled`:** Run a method on a schedule:
  - `fixedRate` — every N ms (regardless of execution time)
  - `fixedDelay` — N ms after the previous execution completes
  - `cron` — cron expression (`@Scheduled(cron = "0 0 2 * * ?")` = 2 AM daily)
- **Thread pool:** `ThreadPoolTaskExecutor` for async; `ThreadPoolTaskScheduler` for scheduling.

Common Questions

- **How does `@Async` work?** Spring proxies the bean. When called, the proxy submits the method to a `TaskExecutor` thread pool instead of running it inline. **Self-invocation breaks it** (same as `@Transactional`).
- **`fixedRate` vs `fixedDelay`?** `fixedRate` runs every N ms from the start time (may overlap). `fixedDelay` waits N ms after completion (no overlap).
- **What is `CompletableFuture`?** Java's async programming abstraction. `@Async` methods can return `CompletableFuture<T>` for non-blocking composition.

---

18. Spring Boot Starters & Dependencies

Key Concepts

- **Starters:** Opinionated dependency descriptors that bundle related libraries.
  - `spring-boot-starter-web` — Spring MVC + embedded Tomcat + Jackson
  - `spring-boot-starter-data-jpa` — Spring Data JPA + Hibernate
  - `spring-boot-starter-validation` — Bean Validation + Hibernate Validator
  - `spring-boot-starter-test` — JUnit 5 + Mockito + AssertJ + Spring Test
  - `spring-boot-starter-actuator` — production monitoring endpoints
  - `spring-boot-starter-security` — Spring Security
- **`spring-boot-starter-parent`:** Provides dependency management (versions), plugin config, Java version.
- **BOM (Bill of Materials):** `spring-boot-dependencies` POM — all managed versions in one place.
- **Scope:** `compile` (default), `runtime` (not at compile, at runtime), `test`, `provided`, `optional`.

Common Questions

- **What is a Spring Boot starter?** A POM that bundles commonly-used dependencies for a feature so you add one dependency instead of many.
- **What does `spring-boot-starter-web` include?** Spring MVC, embedded Tomcat, Jackson (JSON), validation starter.
- **What is the purpose of `optional = true`?** (Lombok in this project) The dependency is available at compile time but won't be transitively included in dependents. Lombok is only needed at compile time (annotation processing), not at runtime.
- **`runtime` scope?** The dependency is not needed at compile time but is at runtime. H2 in this project uses `runtime` scope — your code doesn't import H2 classes directly.

Where in this project

- `pom.xml` — all starters and dependencies with their scopes

---

19. Build & Packaging

Key Concepts

- **Maven lifecycle phases:** `validate` → `compile` → `test` → `package` → `verify` → `install` → `deploy`
- **`spring-boot-maven-plugin`:** Packages the app as an executable JAR (fat JAR) with an embedded Tomcat.
- **Fat JAR:** A single JAR containing all dependencies + embedded server. Run with `java -jar app.jar`.
- **Layers (Spring Boot 2.3+):** JAR split into layers for efficient Docker images (dependencies layer vs code layer).
- **Profiles in Maven:** `<profiles>` for build-time configuration (e.g., different deps for dev vs prod).

Common Questions

- **How to build an executable JAR?** `mvn clean package` → `target/demo-0.0.1-SNAPSHOT.jar` → `java -jar target/demo-0.0.1-SNAPSHOT.jar`
- **What is a fat JAR?** A JAR with all dependencies embedded. Spring Boot's `spring-boot-maven-plugin` repackages the JAR so it can run standalone.
- **How does the fat JAR run?** `JarLauncher` (Spring Boot's main class in the repackaged JAR) sets up the classpath and calls your `DemoApplication.main()`.
- **Maven vs Gradle?** Both are build tools. Maven is XML-based, declarative, convention-over-configuration. Gradle is Groovy/Kotlin DSL, more flexible, faster with incremental builds.

Where in this project

- `pom.xml` — `spring-boot-maven-plugin` configured (excludes Lombok from the fat JAR)
- `Dockerfile` — multi-stage build: Maven build → JRE runtime

---

20. Design Patterns in Spring

Key Concepts

- **Singleton:** Beans are singletons by default. One instance per container.
- **Factory:** `BeanFactory` / `ApplicationContext` is a factory that creates beans.
- **Proxy:** `@Transactional`, `@Async`, `@Cacheable` work via CGLIB/JDK proxies.
- **Template Method:** `JdbcTemplate`, `RestTemplate`, `RedisTemplate` — define the skeleton, you override specific steps.
- **Dependency Injection:** The core pattern of Spring (a form of Inversion of Control).
- **Observer:** Application events (`ApplicationEventPublisher` + `@EventListener`).
- **Front Controller:** `DispatcherServlet` is the single entry point for all requests.
- **Adapter:** `HandlerAdapter` adapts different controller types to a common interface.
- **Decorator:** `BeanPostProcessor` can wrap/modify beans after creation.

Common Questions

- **Which design patterns does Spring use?** Singleton, Factory, Proxy, Template Method, Dependency Injection, Observer, Front Controller, Adapter, Decorator.

- **How does the proxy pattern work in Spring?** When you add `@Transactional` or `@Async`, Spring creates a proxy (CGLIB subclass or JDK dynamic proxy). The proxy intercepts calls and adds behavior (begin/commit transaction, submit to thread pool) before delegating to the real method.
- **`BeanPostProcessor`?** A hook to modify beans after instantiation but before they're used. Used internally by Spring for `@Autowired`, `@Transactional`, etc.

---

21. Java Core Topics Often Asked

These come up frequently in Spring Boot interviews because Spring builds on them:

- **Collections:** `HashMap` internals (buckets, red-black trees, load factor), `ArrayList` vs `LinkedList`, `ConcurrentHashMap`
- **Concurrency:** `volatile`, `synchronized`, `ReentrantLock`, `CompletableFuture`, `ExecutorService`, thread pools, `ThreadLocal`
- **Java 8 Features:** Lambdas, `Stream API` (`map`, `filter`, `reduce`, `collect`), `Optional`, `Function/Predicate/Consumer`, method references
- **Java 17 Features:** Records, sealed classes, pattern matching, switch expressions, text blocks
- **Garbage Collection:** Generational GC, GC algorithms (G1, ZGC), memory model (heap, stack, metaspace)
- **Exceptions:** Checked vs unchecked, exception hierarchy, try-with-resources, custom exceptions
- **Generics:** Type erasure, wildcards (`? extends`, `? super`), PECS (Producer Extends, Consumer Super)
- **I/O:** `InputStream/OutputStream`, NIO, `Files` API
- **Annotations:** Retention policies (`SOURCE`, `CLASS`, `RUNTIME`), annotation processing (Lombok uses this)

Streams in this project

- `UserService.getAllUsers()` — `.stream().map(this::toResponse).toList()`
- `GlobalExceptionHandler.handleValidation()` — `.stream().map(fe -> ...).toList()`

---

22. Quick-Fire Questions

Short answers to memorize:

Question	Answer
Spring vs Spring Boot?	Spring is a framework; Spring Boot is an opinionated layer that auto-configures Spring, embeds a server, and reduces boilerplate.
`@Component` vs `@Bean`?	`@Component` annotates a class; `@Bean` annotates a method (in `@Configuration`).
`@Autowired` vs `@Resource`?	`@Autowired` is Spring (by type); `@Resource` is JSR-250 (by name).
`@RequestParam` vs `@PathVariable`?	Query string vs URL path segment.
`@RequestBody` vs `@ModelAttribute`?	JSON body (REST) vs form data (MVC).
`@Controller` vs `@RestController`?	`@RestController` = `@Controller` + `@ResponseBody`.

Question	Answer
`@Configuration` vs `@Component`?	`@Configuration` proxies `@Bean` methods for singleton semantics.
`@Transactional` rollback rule?	Rolls back on unchecked exceptions only (unless `rollbackFor` is set).
`ddl-auto` for production?	`validate` or `none` — never `create`/`update`.
`JpaRepository` vs `CrudRepository`?	`JpaRepository` adds `flush()`, batch operations, and JPA-specific methods.
`findById` return type?	`Optional<T>`.
`save()` for new vs existing?	New → `persist()`; existing (detached) → `merge()`.
First-level cache?	Persistence context (per transaction).
Second-level cache?	Cross-session, shared, needs explicit config.
Default connection pool?	HikariCP.
Default embedded server?	Tomcat.
How to disable auto-config?	`@SpringBootApplication(exclude = ...)`.
`@Value` vs `@ConfigurationProperties`?	Single value vs type-safe group binding.
Bean scope default?	Singleton.
`@Valid` vs `@Validated`?	Jakarta Validation (fields) vs Spring (groups + method-level).
Fat JAR?	Single JAR with all deps + embedded server.
`@Async` limitation?	Self-invocation bypasses the proxy (no async).
`@Transactional` limitation?	Self-invocation bypasses the proxy (no transaction).

Running This Project

Prerequisites

- Java 17 (managed via SDKMAN)
- Maven 3.9+ (managed via SDKMAN)

Run

```
mvn spring-boot:run
```

Build a JAR

```
mvn clean package
java -jar target/demo-0.0.1-SNAPSHOT.jar
```

Run with Docker

```
docker build -t spring-boot-learning .
docker run -p 8080:8080 spring-boot-learning
```

Try the API

```
# Hello endpoint
curl http://localhost:8080/hello?name=Spring

# Create a user
curl -X POST http://localhost:8080/api/users \
  -H "Content-Type: application/json" \
  -d '{"name":"Alice","email":"alice@example.com","age":30}'

# Get all users
curl http://localhost:8080/api/users

# Get user by id
curl http://localhost:8080/api/users/1

# Update user
curl -X PUT http://localhost:8080/api/users/1 \
  -H "Content-Type: application/json" \
  -d '{"name":"Alice Updated","email":"alice2@example.com","age":31}'

# Delete user
curl -X DELETE http://localhost:8080/api/users/1

# Validation error (empty name)
curl -X POST http://localhost:8080/api/users \
  -H "Content-Type: application/json" \
  -d '{"name":"","email":"bad"}'

# H2 console
# http://localhost:8080/h2-console (JDBC URL: jdbc:h2:mem:testdb, user: sa, no password)
```

Project Structure

```
src/main/java/com/learn/demo/
├── DemoApplication.java          # Entry point (@SpringBootApplication)
├── controller/
│   ├── HelloController.java      # Simple REST endpoint
│   └── UserController.java       # CRUD REST API with DTOs
├── service/
│   └── UserService.java          # Business logic + @Transactional
├── repository/
│   └── UserRepository.java       # Spring Data JPA interface
├── model/
│   └── User.java                 # JPA entity with validation
├── dto/
│   ├── UserRequest.java          # Request DTO (input validation)
│   └── UserResponse.java         # Response DTO (no internal fields)
└── exception/
    ├── GlobalExceptionHandler.java  # @RestControllerAdvice
    ├── ResourceNotFoundException.java
```

```
    ├── EmailAlreadyExistsException.java
    └── ErrorResponse.java        # Structured error response body

src/main/resources/
└── application.yml               # Configuration

src/test/java/com/learn/demo/
└── DemoApplicationTests.java     # Smoke test (@SpringBootTest)
```